

Dangling Else Grammar

Using `matched_stmt` and `unmatched_stmt` with examples, derivations, and parse trees

Core idea: make every else bind to the nearest unmatched if

The Problem: Dangling Else

A nested if without braces can be parsed in more than one way by the naïve grammar.

```
if E1 then
    if E2 then
        S1
    else
        S2
```

Ambiguous source form

Parse A
else matches inner if
E2



Conventional rule
else binds to nearest
unmatched if

Parse B
else matches outer if
E1



Disallowed by the
matched/unmatched
grammar

Naïve Grammar Creates Ambiguity

The standard compact grammar does not force the nearest-if rule.

Naïve grammar

```
S → if E then S  
   | if E then S else S  
   | other
```

One input string

Two parse trees

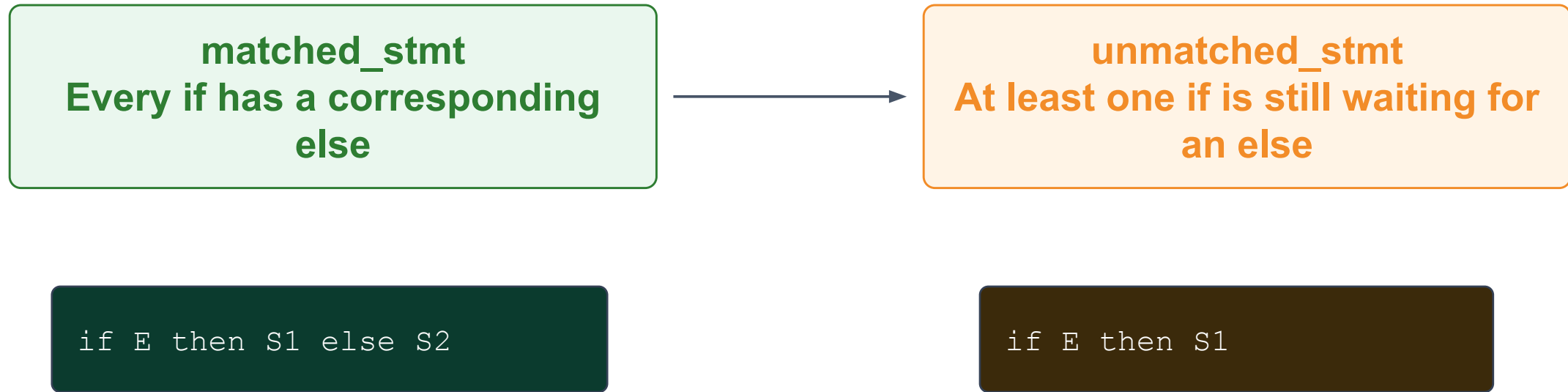
```
if E1 then if E2 then S1 else S2
```

Ambiguous grammar is unsuitable for deterministic parsing tables.

Ambiguity means two valid parse trees for the same token sequence.
Parser generators may report conflicts, such as shift/reduce conflict.
We can remove ambiguity by classifying statements.

Key Idea: Matched vs Unmatched Statements

We split statements by whether every if already has its else.



Matched statements are complete.
Unmatched statements carry the dangling-if position.
This structure enforces the nearest-if rule grammatically.

The Unambiguous Grammar

This is the standard grammar transformation for resolving dangling else.

Grammar rules

```
stmt → matched_stmt | unmatched_stmt
```

```
matched_stmt → if E then matched_stmt else matched_stmt  
              | other_statement
```

```
unmatched_stmt → if E then stmt  
                | if E then matched_stmt else unmatched_stmt
```

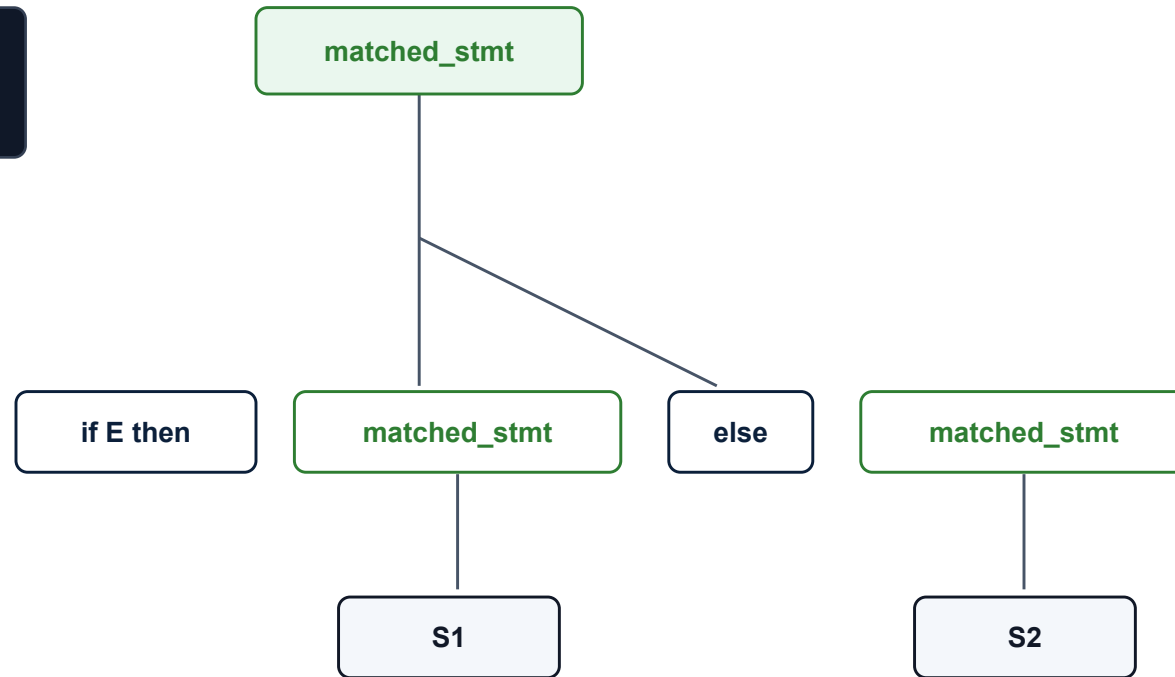
The second unmatched rule handles an else whose right side is still incomplete.

Example 1: A Fully Matched if-else

Input: if E then S1 else S2

```
if E then S1 else S2
```

This is `matched_stmt` because the if has a then-part and an else-part.



Derivation for Example 1

Apply matched_stmt production and replace both branches with other_statement.

Target: `if E then S1 else S2`

Result: complete / matched

`stmt`



`⇒ matched_stmt`



`⇒ if E then matched_stmt else matched_stmt`



`⇒ if E then other_statement else other_statement`



`⇒ if E then S1 else S2`

Example 2: The Dangling-Else String

Input: if E1 then if E2 then S1 else S2

```
if E1 then
  if E2 then S1 else S2
```

Outer if E1 has no else $\Rightarrow$ outer statement is unmatched.

Inner if E2 has an else $\Rightarrow$ inner statement is matched.

Correct binding

```
if E1 then {
  if E2 then S1
  else S2
}
```

else S2 belongs to if E2, the nearest unmatched if.

Derivation for Example 2

The outer if is unmatched; its body is a matched inner if-else.

stmt



⇒ unmatched_stmt



⇒ if E1 then stmt



⇒ if E1 then matched_stmt



⇒ if E1 then if E2 then matched_stmt else matched_stmt

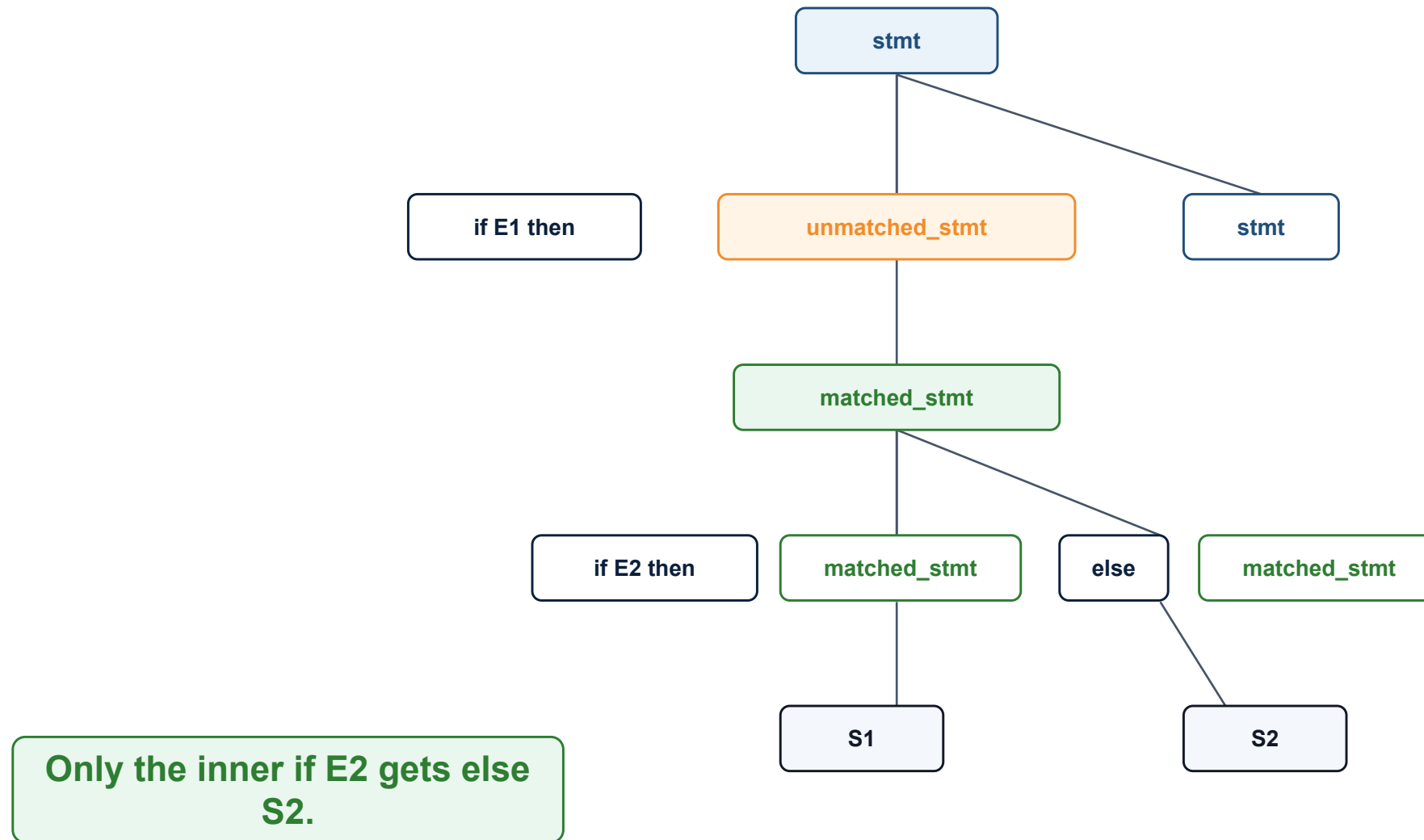


⇒ if E1 then if E2 then S1 else S2

No alternative derivation for outer else

Parse Tree for Example 2

The grammar makes the intended parse tree the only valid one.



Example 3: Else-if with an Unmatched Tail

Input: if E1 then S1 else if E2 then S2

```
if E1 then S1
else if E2 then S2
```

Outer if has else, but its else-branch is an unmatched statement.

Use this rule:

unmatched production

```
unmatched_stmt → if E then matched_stmt
                  else unmatched_stmt
```

```
if E1 then S1 else {
    if E2 then S2
}
```

Derivation for Example 3

The second unmatched production is essential here.

stmt



⇒ unmatched_stmt



⇒ if E1 then matched_stmt else unmatched_stmt



⇒ if E1 then S1 else unmatched_stmt



⇒ if E1 then S1 else if E2 then stmt



⇒ if E1 then S1 else if E2 then S2

Meaning: the outer else is present, but the else-branch still contains a dangling if.

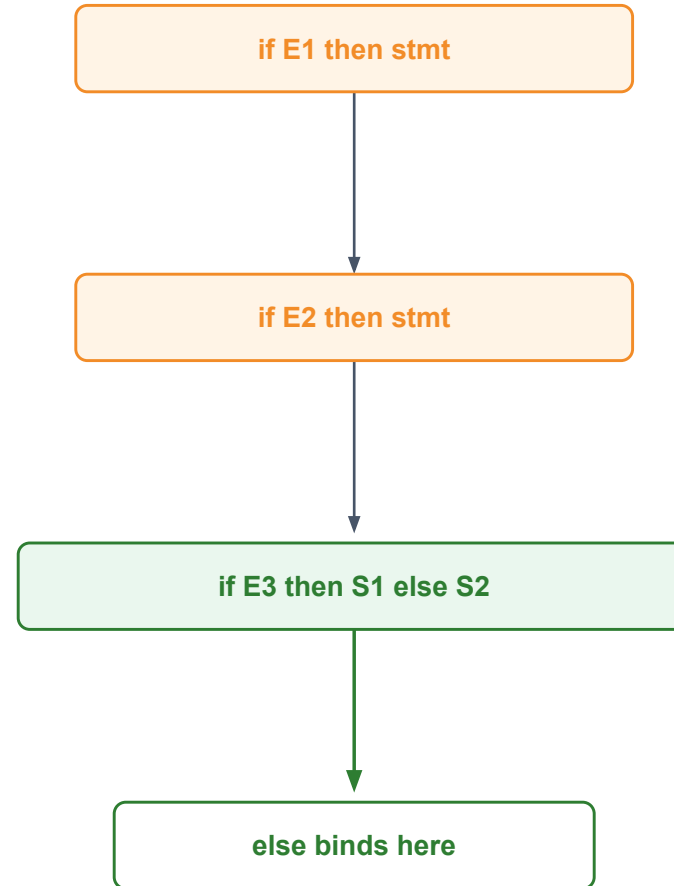
Example 4: Multiple Nested if Statements

Input: if E1 then if E2 then if E3 then S1 else S2

```
if E1 then
  if E2 then
    if E3 then S1 else S2
```

Nearest-if rule

```
if E1 then {
  if E2 then {
    if E3 then S1
    else S2
  }
}
```



E1 and E2 remain unmatched; E3 is matched.

How the Grammar Prevents the Wrong Parse

The grammar disallows attaching else to a farther if when a nearer unmatched if exists.

Wrong interpretation

```
if E1 then {  
    if E2 then S1  
}  
else S2
```

Why invalid?

To attach else to E1, the then-part of E1 must be matched. But the then-part is “if E2 then S1”, which is unmatched. Therefore, this derivation cannot be built.

Correct interpretation

```
if E1 then {  
    if E2 then S1  
    else S2  
}
```

Summary + Classroom Exercise

Use matched/unmatched classification to make every parse deterministic.

3 Takeaways

matched_stmt: every if has an else.
unmatched_stmt: some if is still waiting.
else always attaches to the nearest unmatched if.

Try this derivation

```
if E1 then
  if E2 then S1
  else if E3 then S2 else S3
```

Question: Which if does each else match?

Answer hint: start from stmt → unmatched_stmt, then classify each inner statement.